

2차시 · Codex 실전

Codex 실전 개발법: 컨텍스트 엔지니어링

같은 모델, 다른 결과 무엇을 보여줄지가 품질을 만든다.
OpenAI 팀이 실제로 Codex를 쓰는 방식 그대로 배웁니다.

SECTION 00

오프닝

코드 생성기는 출력을 만들고, Codex는 결과를 만든다.

“왜 어떤 사람은 같은 AI로 훨씬 더 좋은 결과를 낼까?”

답: 프롬프트가 아니라 맥락(context) 설계

이미 현실이 된 일

OpenAI 팀이 Codex로 직접 만든 것

28일

Android용 Sora 앱 출시 (4명)

50억

소비 토큰 · 99.9% crash-free

5개월

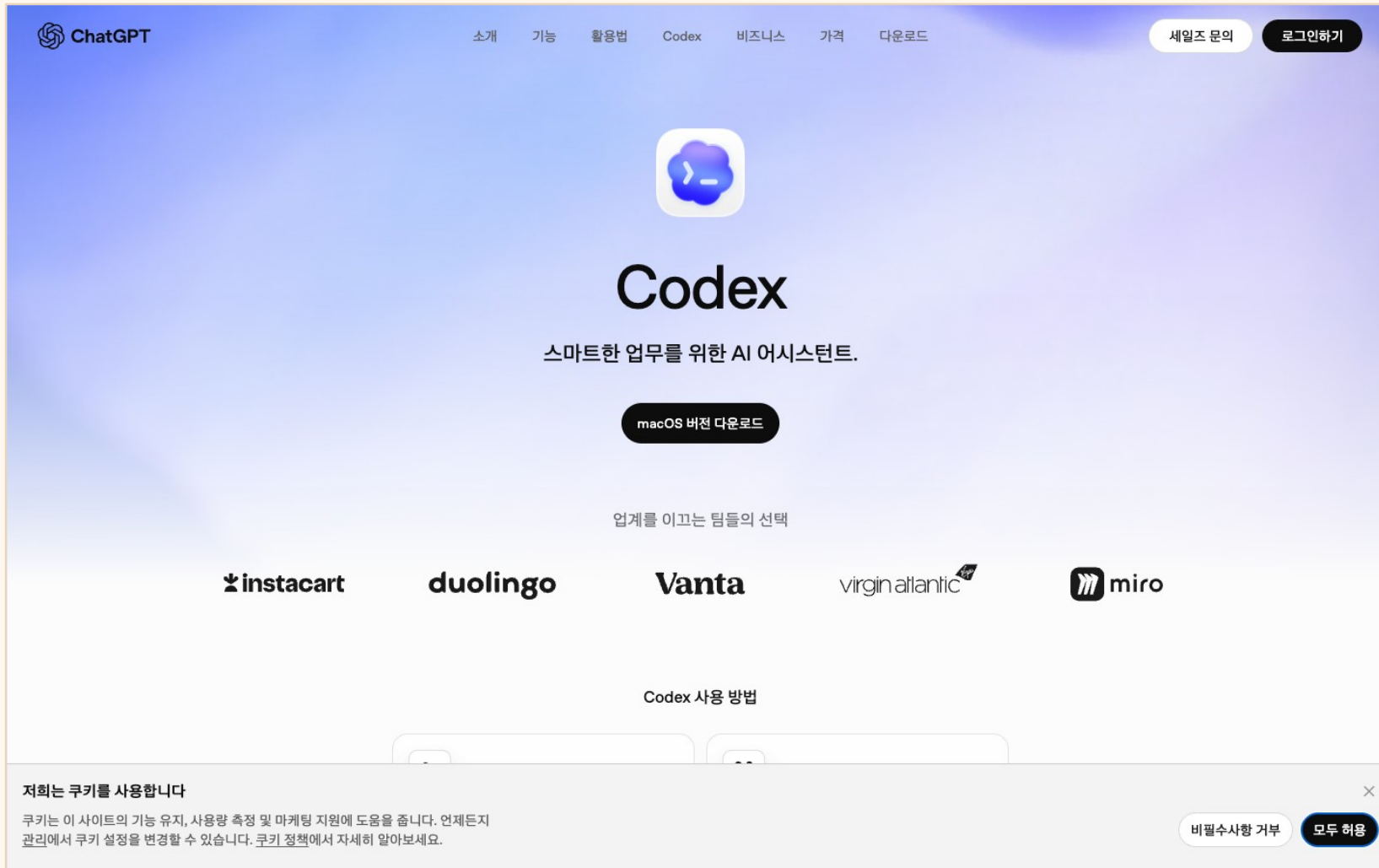
“수동 코드 0줄”로 내부 제품 구축

수작업 대비 약 1/10 시간. 4일 안에 미니 서비스를 만드는 건 결코 무리한 목표가 아닙니다.

실제 화면

chatgpt.com/codex — Codex 공식 페이지

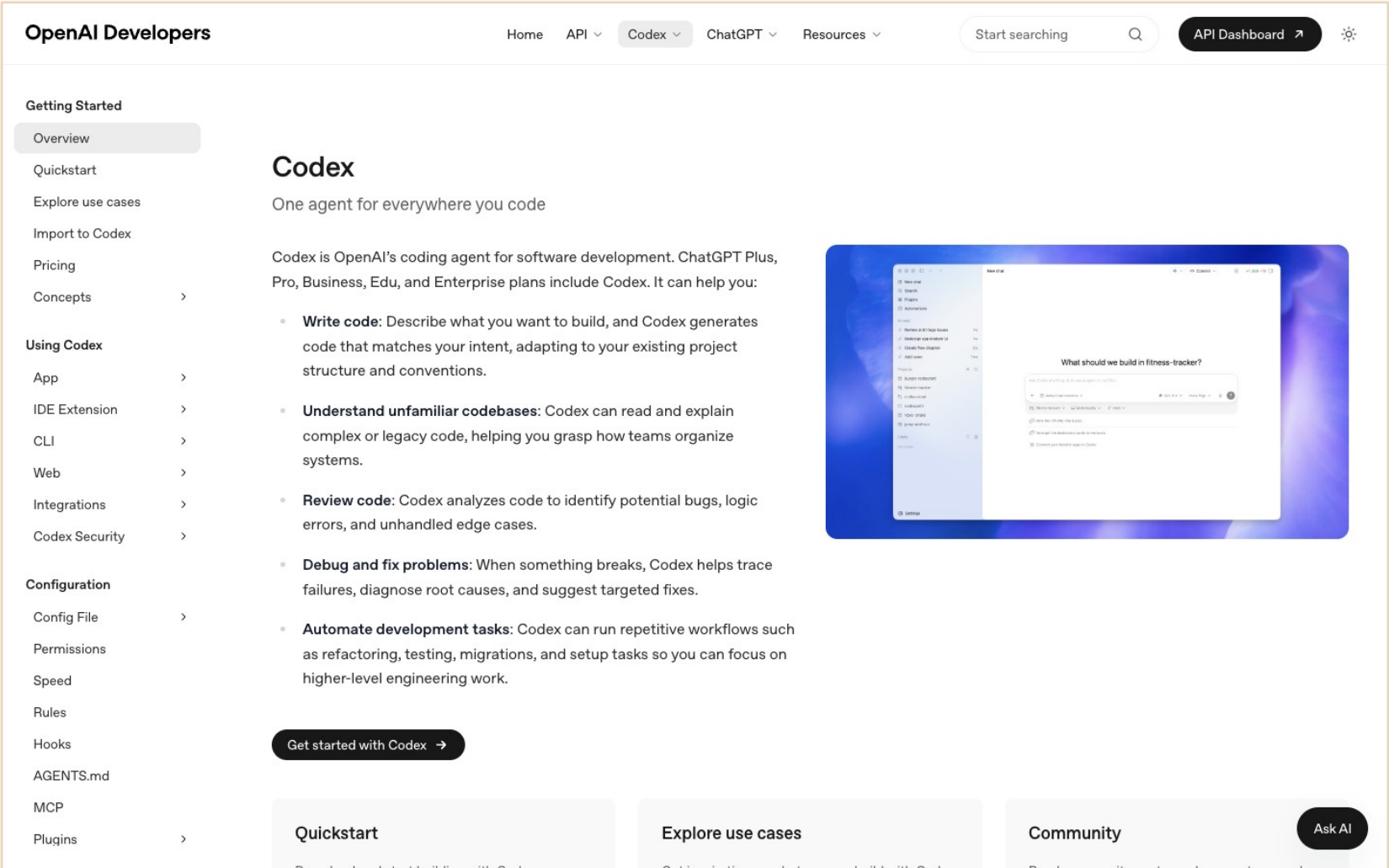
웹·데스크톱 앱·CLI로 쓰는 Codex — 다운로드도 여기서



실제 화면

developers.openai.com/codex — 개발자 문서

CLI·IDE·클라우드 등 모든 사용법의 공식 레퍼런스



ChatGPT 붙여넣기 vs Codex 위임

ChatGPT (매 단계의 다리)

- 내가 코드를 받아 직접 붙여넣고 연결
- 맥락은 매번 내가 다시 설명
- 루프(반복)를 내가 소유
- 파일 실행·검증은 내 몫

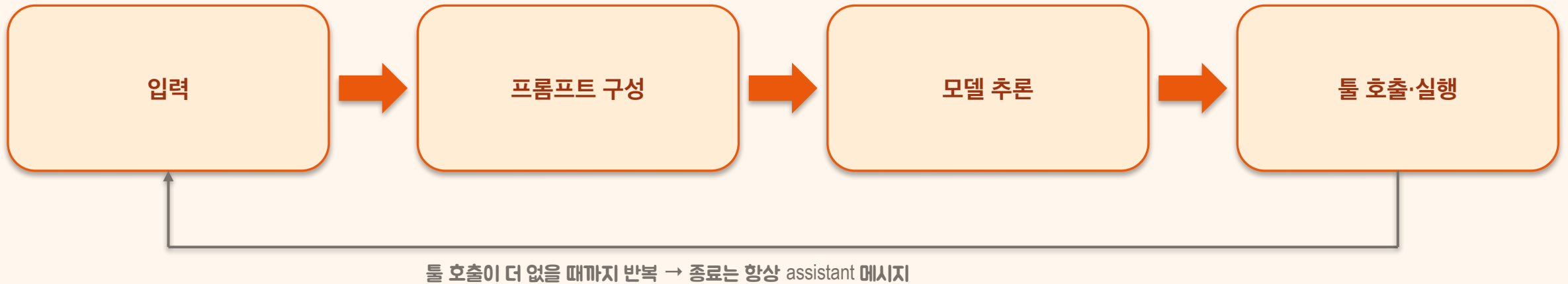
Codex (위임하고 검토)

- 목표를 주면 에이전트가 실행
- AGENTS.md로 맥락을 한 번만 정의
- 루프를 에이전트가 소유, 나는 리뷰
- 파일 편집·테스트·커밋까지 한 흐름

SECTION 01

에이전트 루프 & 셋업

Codex는 한 턴 안에서 이렇게 돈다



한 턴(=스레드) 안에 추론·툴 호출이 수십~수백 번. Responses API HTTP 요청으로 실제로 돌아간다.

config.toml로 모델·자율성의 경계를 고정

~/.codex/config.toml 또는 프로젝트 내에서 .codex/config.toml

- model — 작업에 쓸 모델
- approval_policy — 변경 승인 시점 (기본: on-request)
- sandbox_mode — 파일 쓰기 범위 (기본: workspace-write)
- 웹 검색·프로필·verbosity도 파일로 “에이전트가 일하는 조건”을 내 손에

```
model = "gpt-5-codex"  
approval_policy = "on-request"  
sandbox_mode = "workspace-write"
```

SECTION 02

AGENTS.md = README-first

에이전트가 읽을 수 없는 것은 사실상 존재하지 않는다.

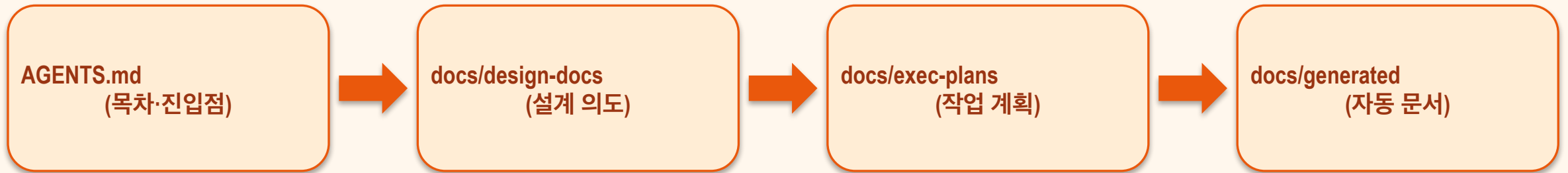
AGENTS.md = 에이전트에게 주는 프로젝트 설명서

▶ 루트에 두면 매번 자동 참고 — 규칙을 프롬프트에 다시 적지 않아도 적용된다



가장 중요한 구분

AGENTS.md = 목차(thin), docs/ = 본문



거대한 단일 AGENTS.md는 실패한다 컨텍스트는 희소 자원이고, 지침이 너무 많으면 지침이 되지 않는다.
짧게(~100줄) 유지하고 지식은 버전관리되는 docs/(기록 시스템)에 저장한다.

코딩 전에 Codex와 함께 계획하기

1

먼저 이해를 요청

“이 시스템이 어떻게 작동하는지 먼저 설명해줘” 바로 코드 X

2

그 이해를 수정·보완

틀린 가정을 잡고, 빠진 맥락을 채운다

3

작은 설계 문서 작성

합의된 계획을 docs/에 남긴다 (공동 로드맵)

4

한 단계씩 적용

계획을 뒷받침하는 코드로 큰 추측성 변경 금지

SECTION 03

좋은 프롬프트 — 목표를 안내하라

지나치게 세세한 지시는 오히려 결과를 악화시킨다.

좋은 프롬프트의 일반 원칙

Less is More

- 전체 툴을 다 노출하면 혼란
- 겹치는 도구는 제한·통합
- → MCP `enabled_tools`로 최소화

Guide the Goal

- 경로가 아니라 목표를 줘라
- 세세한(prescriptive) 지시 = 악화
- 높은 수준 방향 → 추론에 맡김

Meaning Lives in Code

- 의미는 코드에 산다
- 스키마·이력만으론 부족
- 코드를 크롤링해 진짜 의미 도출

같은 프롬프트, 다른 결과

컨텍스트 없음

- “설정 화면 만들어줘”
- 평범·제각각인 결과
- 스타일·구조를 매번 다시 지정

AGENTS.md + 예시

- “방금 본 화면과 같은 패턴으로”
- 규칙에 맞춘 일관된 결과
- 검증 기준까지 충족

SECTION 04

컨텍스트 엔지니어링 — 맥락이 전부

메모리 없는 에이전트는 22분을 헤매고, 맥락이 있으면 1분 22초에 정답.

무엇을 컨텍스트에 넣고 빼는가

① 테이블 / 코드 사용
에이전트가 직접 읽는 1차 소스

② 사람 주식
왜 이렇게 했는지 의도

④ 조직 지식
팀의 규칙·관례

⑤ 메모리
과거 작업의 누적

⑥ 런타임 컨텍스트
지금 이 작업의 입력

정적 콘텐츠는 프롬프트 앞, 변하는 건 뒤 → 프롬프트 캐시 적중↑.

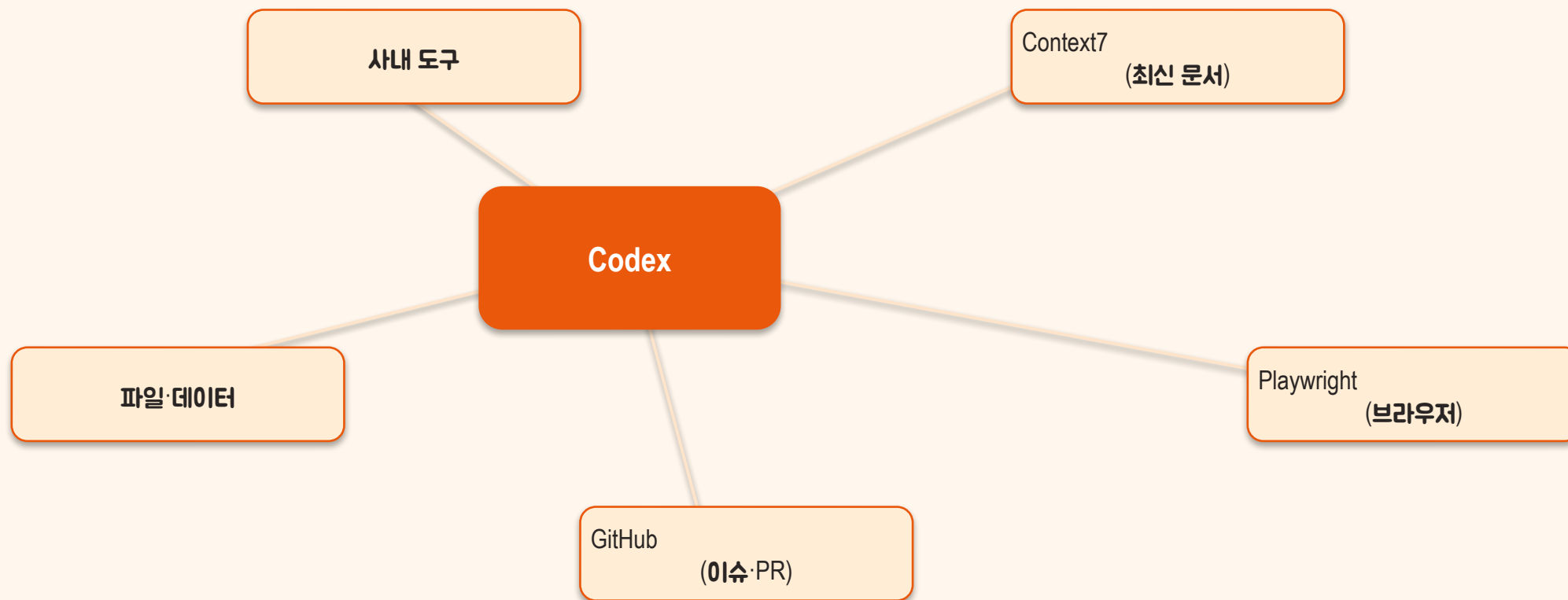


SECTION 05

도구 확장 — 플러그인 · MCP · 스킬

플러그인이 상위 개념 스킬·앱·MCP를 한 번에 묶어 설치한다.

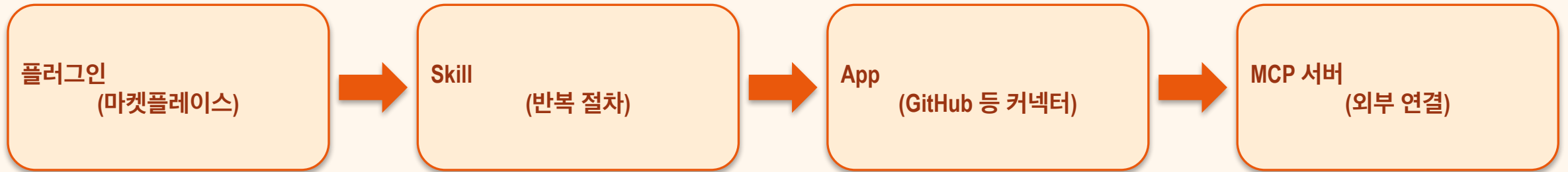
MCP = 에이전트의 손발을 잇는 표준



저장소 밖 정보를 잇는 표준 약속(USB-C 비유). `/mcp`로 확인.

플러그인 = 상위 개념

플러그인이 Skill·App·MCP를 묶는다



탐색·설치: `codex /plugins` · 마켓 추가: `codex plugin marketplace add owner/repo`. 새 플러그인은 권한·훅(`/hooks`)을 먼저 확인.

반복 워크플로를 SKILL.md로 패키징 (4단계)

1

폴더 만들기
`my-skill/`

2

SKILL.md 작성
프론트매터 `name · description`(언제 쓰는지 명확히 자동 호출의 핵심)

3

보조 자료 추가(선택)
`scripts/ · references/ · assets/`

4

위치에 저장 → 호출
`.codex/skills/` 또는 `~/.codex/skills/` → `$skill-name` 또는 `/skills`

ponytail — “안 쓴 코드가 최고의 코드”

~54%

평균 코드량 감소(최대 94%)

lite → full → ultra

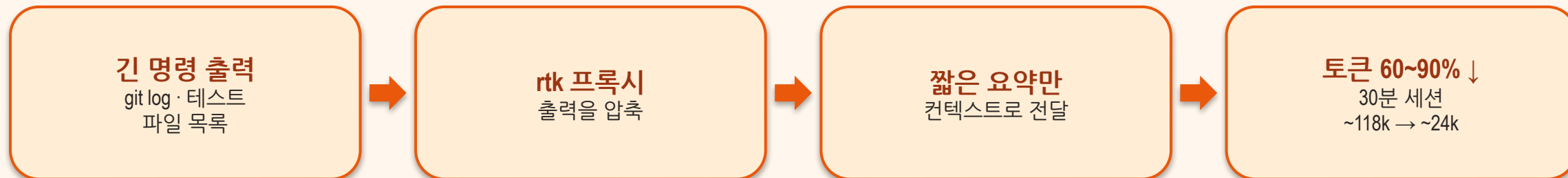
모드 3단계 — 권고 → 사다리 강제 → 한 줄 미니멀

설치: `codex plugin marketplace add DietrichGebert/ponytail` → `/plugins` 설치 → `/hooks` 검토 → 새 스레드

스킬 `@ponytail-review@ponytail-audit@ponytail-gain` → 실습 Lab G-3: 모드별 피보나치 구현 비교

긴 세션의 토큰을 60~90% 절약

▶ MCP가 아닌 셸 프록시 — 설치 후 자동 적용, rtk gain 으로 절감 확인



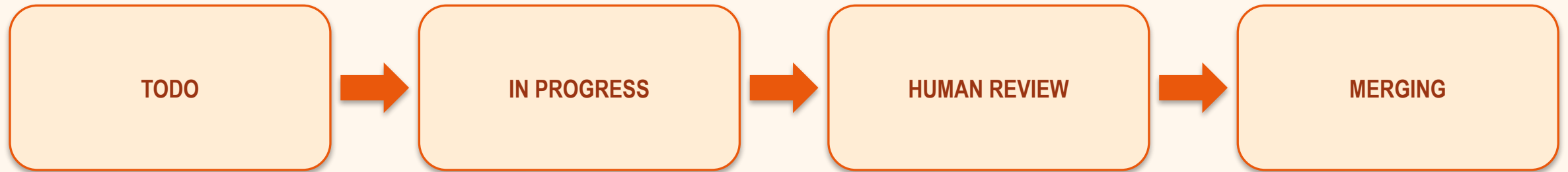
AGENTS.md에 “검색·로그·테스트는 rtk를 써라” 규칙을 넣으면 에이전트가 자동으로 사용

SECTION 06

협업 & 확장 — 오케스트라 지휘자

솔로 연주자가 아니라, 여러 세션을 조율하는 지휘자가 된다.

이슈 트래커 = 에이전트 오케스트레이터



에이전트가 작업 트리(DAG)를 만들고 막히지 않은 일부터 병렬 실행. 핵심은 **SPEC.md**(문제+해법)·**WORKFLOW.md**(정책). → 실습 Lab J에서 미니 보드로 직접 재현.

SECTION 07

실습 & 안전

빠를수록 더 신중하게 검토한다.

실습 노트북 2차시_실습.ipynb — Lab A부터 J까지

1

Lab A~C · 프롬프트 기법 눈으로 확인

5요소(Goal·User·Behavior·Constraints·Success) · JSON/Few-shot · CoT

2

Lab D~E · 컨텍스트가 결과를 바꾼다

AGENTS.md 초안 생성 · 에러는 요약 말고 원문 그대로 — 에러를 컨텍스트로

3

Lab F~I · 도구 확장

슬래시 / 사전 · outfit-stylist 스킴 → my-style-pack 플러그인 · ponytail 모드 비교 · MCP · rtk

4

Lab J · Symphony 맛보기 → 터미널 마무리

SPEC.md·WORKFLOW.md → 이슈 분해·상태 보드 · AGENTS.md 규칙대로 리팩터링

AI는 도구, 결정과 책임은 사람

▶ 빠를수록 더 신중하게 — 네 가지 안전 습관

① 승인 전 검토

잘 실행돼도 diff를 읽는다 — 타협 불가

② pass-through 권한

내가 이미 권한 있는 것만 에이전트도 접근

③ 외부 데이터 주의

MCP·이슈·문서를 소비한 출력은 더 간간히

④ Git으로 되돌리기

작업은 커밋 단위로 — 언제든지 복구 가능

맥락을 주고, 위임하고, 검토하라.

목표가 명확하고 맥락이 풍부할수록 결과가 신뢰할 만해진다